# Inference: standard errors, singular information, and Monte Carlo sets

Misclassification models make inference harder than the estimation makes it look. The point estimate arrives from a routine likelihood maximization; the uncertainty around it does not, because these likelihoods are prone to two specific pathologies. Both are common, both are detectable, and each calls for a different thing to report.

This vignette covers the three options the package offers, in the order you should try them:

1. **Delta-method standard errors**, which are right when the model is point identified and the solution is interior;
2. **Diagnostics** for when it is not — and the difference between a solution at a boundary and one that is weakly identified;
3. **Monte Carlo confidence sets** in the manner of Chen, Christensen and Tamer, which remain valid when the parameter is only partially identified.

Throughout we use a small simulated example so that the truth is known.

```
set.seed(20240101)
syn <- synthetic_data(J = 3, K = 3, I = 1, sample_size = 20000,
                      dgp_delta = "Record Linkage, independent, 10 - 30%")
tab <- syn$tab[[1]]

beta_true <- Pi_to_beta_inner(c(syn$Pi[[1]]), 1:3, 1:3, 1)
beta_true
#> [1] 0.5912974
```

## Option 1: delta-method standard errors

`misclassifyr()` returns `cov_Pi_mle`, the estimated covariance matrix of $\hat{\Pi}$. It is built by inverting the Fisher information for the $\Pi$ block of the parameter vector and pushing it through the Jacobian of the map from parameters to probabilities. `Pi_to_beta()` then takes one more delta-method step, from $\hat{\Pi}$ to the regression coefficient $\beta$.

```
fit <- misclassifyr(
  tab = tab, J = 3, K = 3,
  X_names = as.character(1:3),
  Y1_names = as.character(1:3),
  Y2_names = as.character(1:3),
  model_to_Delta = model_to_Delta_RL_ind,
  X_vals = 1:3, Y_vals = 1:3,
  mle = TRUE, bayesian = FALSE, makeplots = FALSE
)

out <- Pi_to_beta(X_vals = 1:3, Y_vals = 1:3, mle = TRUE,
                  Pi_mle = fit$Pi_hat_mle, cov_Pi = fit$cov_Pi_mle)

c(beta      = unname(out$beta_hat_mle),
  se        = unname(out$se_beta_mle),
```

```
  ci_lower  = unname(out$beta_hat_mle - 1.96 * out$se_beta_mle),
  ci_upper  = unname(out$beta_hat_mle + 1.96 * out$se_beta_mle),
  truth     = beta_true) |> round(4)
#>     beta       se ci_lower ci_upper    truth
#>   0.6017   0.0060   0.5900   0.6134   0.5913
```

The interval covers the truth — though not by much, which is a fair reflection of how much work a sample of twenty thousand can do here. This is what you would report, **provided** the two diagnostics below come back clean. They are cheap. Look at them every time.

## Option 2: reading the diagnostics

### Is the information matrix invertible?

```
fit$fisher_info_err
#> [1] "Fisher information matrix is invertible."
```

When this reports that the information matrix is singular, `misclassifyr()` falls back to a Moore–Penrose pseudo-inverse and warns you. A pseudo-inverse always returns *a* number; it does not return a variance. The message means the likelihood has at least one direction along which it is flat to second order, and the delta method has nothing to propagate along that direction.

A more informative version of the same check looks at the whole parameter vector, not just the $\Pi$ block, through the eigenvalues of the observed information:

```
information_spectrum <- function(fit) {
  ev <- eigen(-fit$eta_hessian_mle, symmetric = TRUE, only.values = TRUE)$values
  c(smallest = min(ev), largest = max(ev),
    condition = max(ev) / max(min(abs(ev)), .Machine$double.eps))
}

round(information_spectrum(fit), 3)
#>  smallest   largest condition
#>    19.358  3743.614   193.393
```

A healthy fit has all eigenvalues comfortably positive and a condition number in the thousands or less. Values near zero — or, as floating-point arithmetic will sometimes give you, slightly negative — mean a flat direction.

### Does the optimizer find the same answer twice?

```
n_theta <- 3 * 3 + 3^3          # elements of Pi plus elements of Delta
c(total     = fit$inconsistency_mle,
  per_theta = fit$inconsistency_mle / n_theta)
#>        total    per_theta
#> 1.439101e-03 3.997503e-05
```

`misclassifyr()` re-runs the optimization from a perturbed starting value and reports the **total** absolute distance between the two solutions, summed over every element of $\Pi$ and $\Delta$. `check_stability = TRUE` uses nine perturbations instead of one, which makes the statistic more informative and the fit slower.

The package warns above 0.1, which is a useful alarm but a blunt one: it is a sum, so it grows mechanically with the size of the model. Divide by the number of elements before interpreting. On the examples in this vignette the per-element figure separates the cases cleanly — around $4 \times 10^{-5}$ for a healthy fit, $10^{-7}$ when the optimizer converges to a boundary, and several hundred times larger than the healthy case when

identification is weak. Values in that last range mean the likelihood surface has more than one optimum, and a point estimate with a standard error is a misleading summary of what the data say.

### Two very different reasons the information goes singular

The diagnostics above tell you *that* something is wrong. Which of two things it is determines what you should do, and the two look different.

**Boundary solutions.** The true parameter sits on the edge of the parameter space — a false-link rate of exactly zero, a category with zero probability. The likelihood is perfectly well behaved and has a unique optimum; it simply cannot curve in a direction the parameter is not allowed to move. Here is data with no misclassification at all, fitted with a model that allows for it:

```
set.seed(2)
clean <- synthetic_data(J = 3, K = 3, I = 1, sample_size = 5000,
                        dgp_delta = "No error")

fit_boundary <- misclassifyr(
  tab = clean$tab[[1]], J = 3, K = 3,
  X_names = as.character(1:3),
  Y1_names = as.character(1:3),
  Y2_names = as.character(1:3),
  model_to_Delta = model_to_Delta_RL_ind,
  X_vals = 1:3, Y_vals = 1:3,
  mle = TRUE, bayesian = FALSE, makeplots = FALSE
)

c(inconsistency_per_theta = fit_boundary$inconsistency_mle / n_theta,
  smallest_eigenvalue     = information_spectrum(fit_boundary)[["smallest"]])
#> inconsistency_per_theta     smallest_eigenvalue
#>            1.151367e-07           -9.094947e-07
```

The signature: **stable optimizer, singular information.** The optimizer returns the same answer every time — `inconsistency_mle` is essentially zero — but the information matrix has a zero eigenvalue, because the estimated error rate has been pushed to the boundary and the likelihood cannot curve past it. Standard errors on the error parameters are meaningless; the estimate of $\Pi$ itself is fine and, if anything, better than the model deserves. What to do: report the estimate, note that the error rate is at the boundary, and use a confidence set (option 3) rather than a symmetric interval.

**Weak or failed identification.** The likelihood is genuinely flat, or has multiple separated optima, over a region of the parameter space. This is what happens when you ask a small sample to support a rich model — here, the fully unrestricted misclassification model, with $J^2(J-1)$ parameters for $\Delta$, on two thousand observations:

```
set.seed(99)
small <- synthetic_data(J = 3, K = 3, I = 1, sample_size = 2000)

fit_weak <- misclassifyr(
  tab = small$tab[[1]], J = 3, K = 3,
  X_names = as.character(1:3),
  Y1_names = as.character(1:3),
  Y2_names = as.character(1:3),
  model_to_Delta = model_to_Delta_NP,
  log_prior_Delta = log_prior_Delta_NP,
  X_vals = 1:3, Y_vals = 1:3,
  mle = TRUE, bayesian = FALSE, makeplots = FALSE,
```

```
  check_stability = TRUE
)
#> Warning in estimate_misclassification(misclassification_inputs): Optimal theta
#> is inconsistent across starting locations.

c(inconsistency_per_theta = fit_weak$inconsistency_mle / n_theta,
  smallest_eigenvalue     = information_spectrum(fit_weak)[["smallest"]])
#> inconsistency_per_theta     smallest_eigenvalue
#>           0.0194039451          -0.0002386058
```

The signature: **unstable optimizer.** Nine perturbed starts do not agree, and the per-element disagreement is two orders of magnitude above the healthy fit. This is the case where a standard error is not merely imprecise but conceptually wrong: there is no single $\hat{\theta}$ for it to be the standard error *of*. What to do: either restrict the model — a record-linkage or ordinal design with a handful of parameters instead of a saturated one, see `vignette("designing-misclassification-models")` — or report a confidence set that does not presume a unique maximizer.

| Symptom | Inconsistency per element | Smallest eigenvalue | Diagnosis | Report |
|---|---|---|---|---|
| Healthy | negligible | clearly positive | point identified, interior | estimate ± delta-method SE |
| Boundary | negligible | 0 | unique optimum on an edge | estimate + Monte Carlo set |
| Weak / partial | orders of magnitude larger | 0 or negative | flat or multi-modal likelihood | Monte Carlo set only |

## Option 3: Monte Carlo confidence sets

When the parameter may only be partially identified, the object to report is not an interval around a point but a **set** of parameter values the data cannot distinguish. `misclassifyr()` and `Pi_to_beta()` implement the Monte Carlo approach of Chen, Christensen and Tamer: run a Markov chain over the parameter space, keep the draws whose likelihood is within the top 95%, and report the range of $\beta$ over the kept draws.

The logic is worth stating plainly, because it is not the usual Bayesian credible interval. The set is the image, under the map from parameters to $\beta$, of a high-likelihood region of the parameter space. If the likelihood is flat over a ridge, every point on that ridge survives the cut, and the reported set is the whole range of $\beta$ consistent with the data — which is the honest answer. If the likelihood is sharply peaked, the region collapses and the set looks like a conventional interval. The procedure adapts to which situation you are in without your having to decide in advance.

Two things are needed: posterior draws (`bayesian = TRUE`) and the likelihood history along the chain (`ll_history`).

```
fit_b <- misclassifyr(
  tab = tab, J = 3, K = 3,
  X_names = as.character(1:3),
  Y1_names = as.character(1:3),
  Y2_names = as.character(1:3),
  model_to_Delta  = model_to_Delta_RL_ind,
  log_prior_Delta = log_prior_Delta_RL_ind,
  X_vals = 1:3, Y_vals = 1:3,
  mle = TRUE, bayesian = TRUE, makeplots = FALSE,
  n_mcmc_draws = 1500, n_burnin = 500, thinning_rate = 5
)
```

```
#> Warning in misclassifyr(tab = tab, J = 3, K = 3, X_names = as.character(1:3), :
#> `n_mcmc_draws` is too small. Choose a value of at least 10000.
#> Warning in misclassifyr(tab = tab, J = 3, K = 3, X_names = as.character(1:3), :
#> `n_burnin` is too small. Choose a value of at least 5000.
#> Warning in misclassifyr(tab = tab, J = 3, K = 3, X_names = as.character(1:3), :
#> `thinning_rate` is too large. Choose a smaller `n_thinning_rate` or a larger
#> gap between `n_burnin` and `n_mcmc_draws`.
```

The chain length here is deliberately tiny so that this vignette builds quickly; `misclassifyr()` will warn you about it. For anything you intend to report, use at least `n_mcmc_draws = 10000` with `n_burnin = 5000`, and inspect the trace plots (`makeplots = TRUE` returns them in `trace_plots_eta`) before trusting the output.

The acceptance rate is the first thing to check:

```
n_params <- length(fit_b$eta_hat_mle)
c(proposals = 1500 * n_params,
  accepted  = fit_b$accepted_proposals,
  rate      = round(fit_b$accepted_proposals / (1500 * n_params), 3))
#> proposals  accepted      rate
#> 27000.000 12509.000     0.463
```

The sampler updates one coordinate at a time, and the classical target for a one-dimensional random-walk Metropolis step is an acceptance rate near 0.44. Far below that and the proposals are too ambitious: shrink `gibbs_proposal_sd`. Far above and the chain is shuffling: raise it. A rate close to 1 usually means the proposal is so small that the chain has not moved anywhere, which the trace plots will confirm.

Now pass both objects to `Pi_to_beta()`:

```
inference <- Pi_to_beta(
  X_vals = 1:3, Y_vals = 1:3,
  mle = TRUE, bayesian = TRUE,
  Pi_mle       = fit_b$Pi_hat_mle,
  cov_Pi       = fit_b$cov_Pi_mle,
  posterior_Pi = fit_b$posterior_Pi,
  ll_history   = fit_b$ll_history
)

c(mle              = unname(inference$beta_hat_mle),
  delta_se         = unname(inference$se_beta_mle),
  posterior_median = unname(inference$posterior_beta_med),
  posterior_sd     = unname(inference$posterior_beta_sd),
  truth            = beta_true) |> round(4)
#>              mle         delta_se posterior_median     posterior_sd
#>           0.6017           0.0060           0.6022           0.0069
#>            truth
#>           0.5913
```

And the confidence set itself:

```
round(unname(inference$HPDCI), 4)
#> [1] 0.5841 0.6235
length(inference$HPD_draws)   # draws retained out of the post-burn-in sample
#> [1] 191
```

Compare it with the delta-method interval:

```r
delta_ci <- unname(out$beta_hat_mle + c(-1.96, 1.96) * out$se_beta_mle)

knitr::kable(data.frame(
  method = c("Delta method (95%)", "Monte Carlo set"),
  lower  = c(delta_ci[1], inference$HPDCI[1]),
  upper  = c(delta_ci[2], inference$HPDCI[2]),
  width  = c(diff(delta_ci), diff(inference$HPDCI))
), digits = 4)
```

| method | lower | upper | width |
|---|---|---|---|
| Delta method (95%) | 0.5900 | 0.6134 | 0.0234 |
| Monte Carlo set | 0.5841 | 0.6235 | 0.0394 |

On this well-identified example the two agree on the location and differ modestly on the width — the Monte Carlo set is about two-thirds wider, and covers the truth with more room to spare. That is the reassurance you want: the set-valued method costs you some precision when the model is point identified, but not an order of magnitude, and it is the method that keeps working when the delta method quietly stops being valid. Note also that the two are not the same object: the delta-method interval is a 95% confidence interval for a point-identified parameter, while the Monte Carlo set is a 95% confidence set for an identified *set* that here happens to be a single point.

If you omit `ll_history`, `HPD_draws` and `HPDCI` come back as `NA` and you get only the posterior summaries. That is a legitimate thing to report — it is a Bayesian credible interval, interpreted as such — but it is *not* the Chen–Christensen–Tamer set and does not have the same partial-identification guarantee.

## What to report

**Default.** The MLE with a delta-method standard error, having confirmed that `fisher_info_err` reports an invertible information matrix and `inconsistency_mle` is small. State which model of $\Delta$ you used and why; the choice is a modelling assumption on the same footing as a functional form, and readers should be able to see it.

**Add a Monte Carlo set whenever the model is restrictive.** If your design has few enough parameters that you would call it strong, the two intervals will agree and reporting both costs you a sentence. If they disagree, that disagreement is a finding about how much your result depends on the parametric restriction, and the reader deserves to see it.

**Report a set and no standard error when a diagnostic fails.** A confidence set from a flat likelihood is wide, and a wide set is not a failure — it is the correct description of what the data support. Reporting a narrow standard error computed from a pseudo-inverse of a singular matrix, on the other hand, is a claim your data cannot back.

**Always report the error rates you estimated.** $\hat{\Delta}$ is not a nuisance parameter to be hidden; it is a substantive claim about your sources that a reader familiar with those sources can evaluate. If your model says 40% of links failed and the linkage literature for your data says 10%, someone should notice, and it should be you.

## Controls

With control cells (`controls` in `prep_misclassification_data()`), everything above applies within each cell and the outputs become lists. `Pi_to_beta()` aggregates them using `W_weights`, the cell sample sizes, computing $\mathrm{Var}(X)$ and $\mathrm{Cov}(X, Y^*)$ by the law of total variance and covariance rather than by averaging within-cell slopes. It returns both the pooled estimate and the per-cell estimates in `betas_hat_mle`, which are worth inspecting: if the correction behaves very differently across cells, the assumption that errors are unrelated to the regressor may be holding within some cells and not others.

## References

Chen, X., Christensen, T. M., and Tamer, E. (2018). "Monte Carlo Confidence Sets for Identified Sets." *Econometrica* 86(6), 1965–2018.

Mattheis, R. (2024). *Misclassification models for linked historical records.* Working paper.